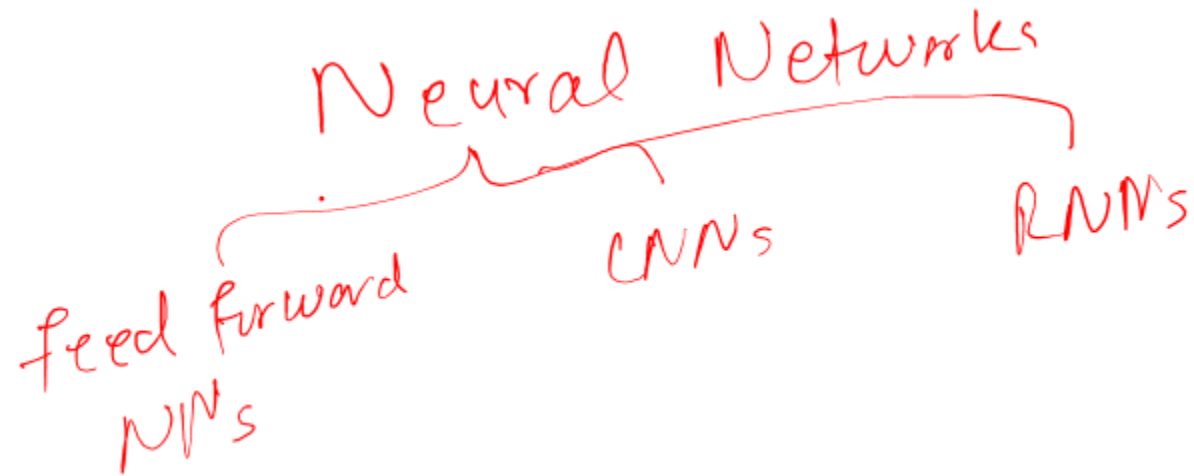




Convolution Neural Networks

09 September 2025



Prepared by Tanveer Hussain (Fellow HEA)
Lecturer, Edge Hill University
htanveer3797@gmail.com

Lecture Outline

- Problems of Neural Networks for Image Classification
- Motivation for CNNs
- Components of CNNs
- Designing a CNN Model

Image Classification

- Neural networks for image classification
- **Problem 1:** The number of parameters become extremely large, causing computation problem and overfitting.

Image Classification

- Neural networks for image classification
- **Problem 2:** Little or no invariance to shifting, scaling, and other forms of distortion.

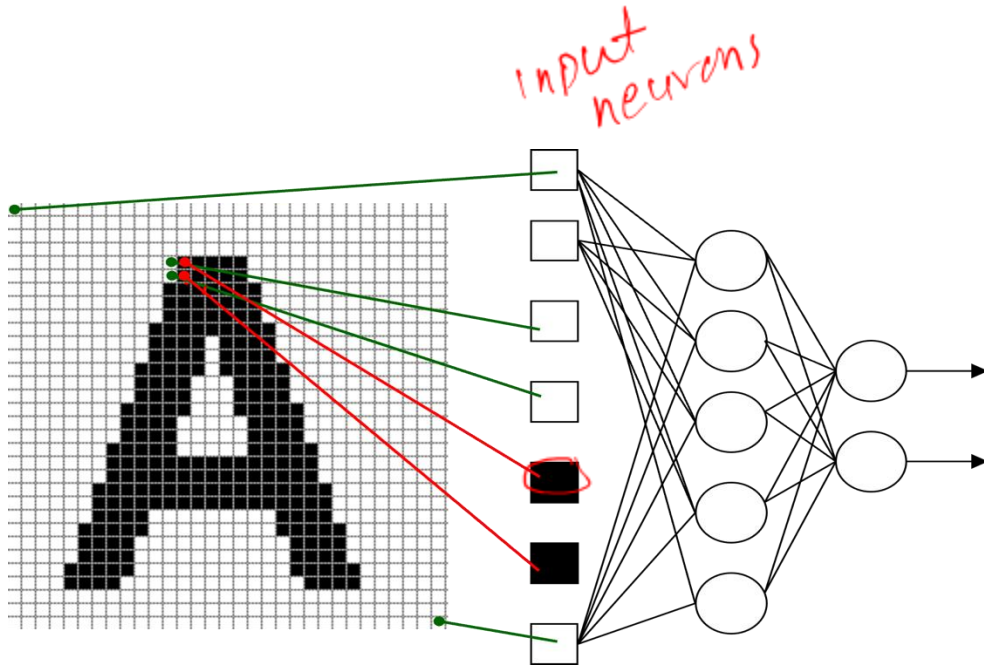


Image Classification

- Neural networks for image classification
- **Problem 2:** Little or no invariance to shifting, scaling, and other forms of distortion.
- **Problem 3:** No local receptive field concept, as each input pixel value is treated independently.

steps → related

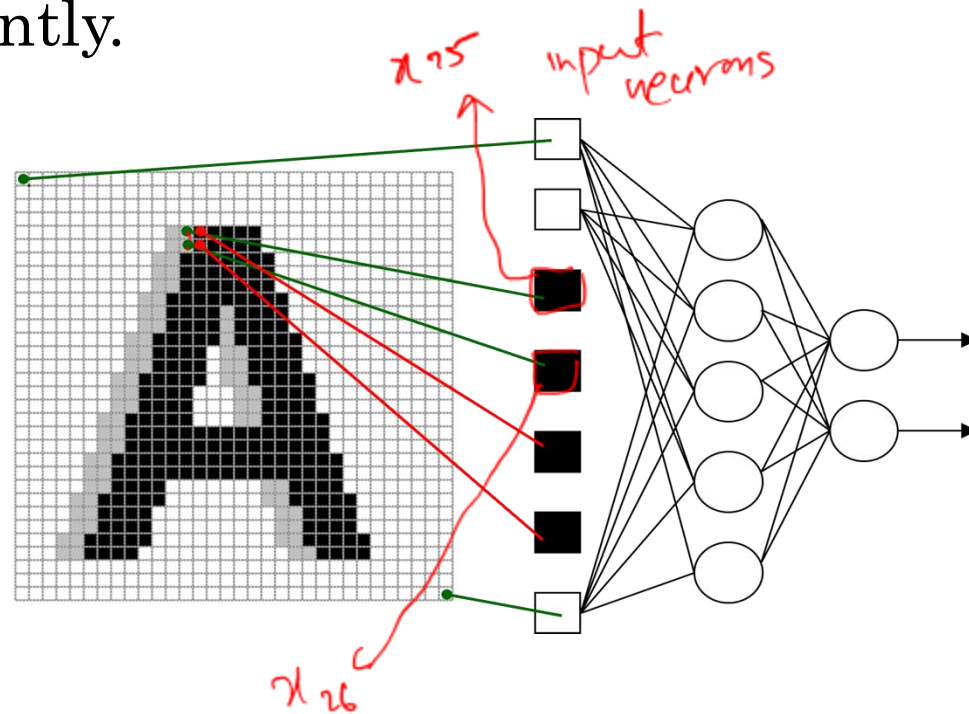
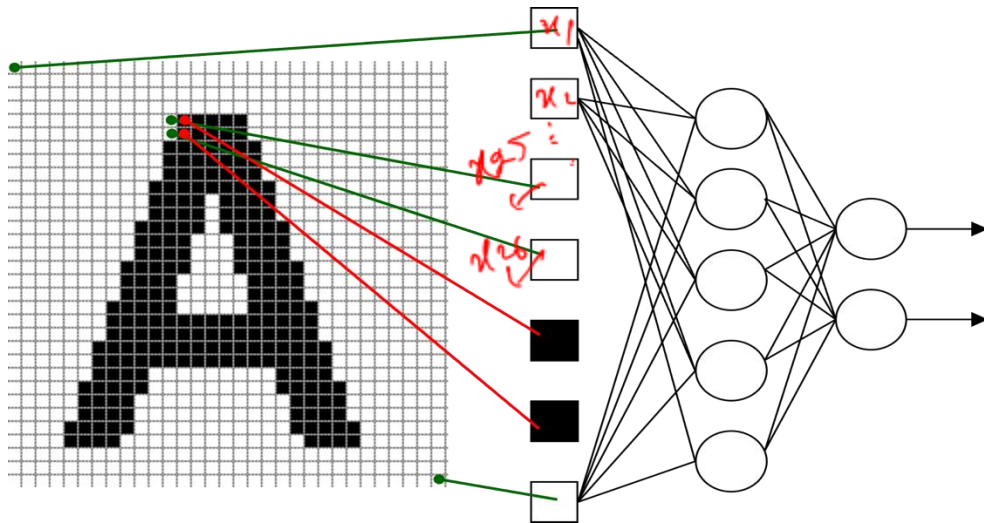


Image Classification

- Neural networks for image classification
- **Problem 4:** The topology of the input data is completely ignored
- For a 32×32 image, we have
 - Black and white patterns: $2^{32 \times 32} = 21024$
 - Grayscale patterns: $256^{32 \times 32} = 2561024$

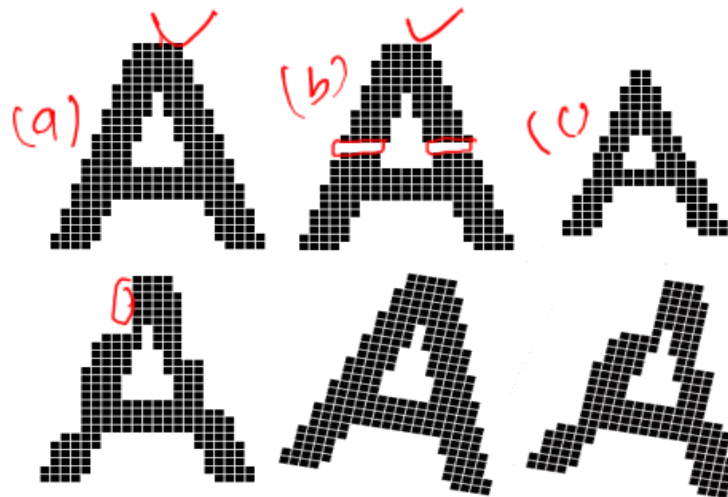


Image Classification



- Image Classification/Visual Recognition model's Expectations
 1. Maintain 2D structure logic
 2. Shift invariant (actually, equivariant)
 3. Consider local correlations first
 - 4. Hierarchically growing field of view
 5. Hierarchically progressing complexity
 - 6. Reasonable amount of params

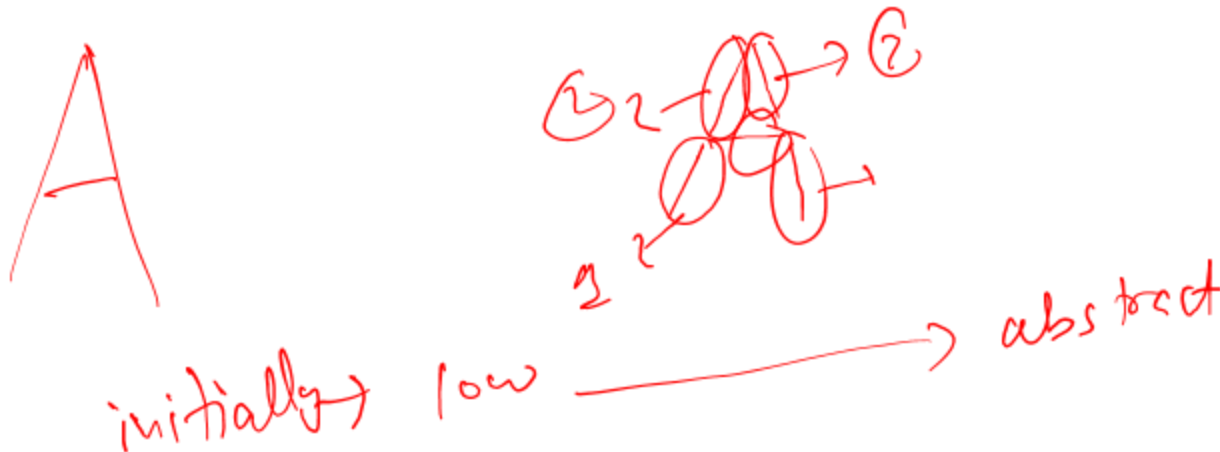


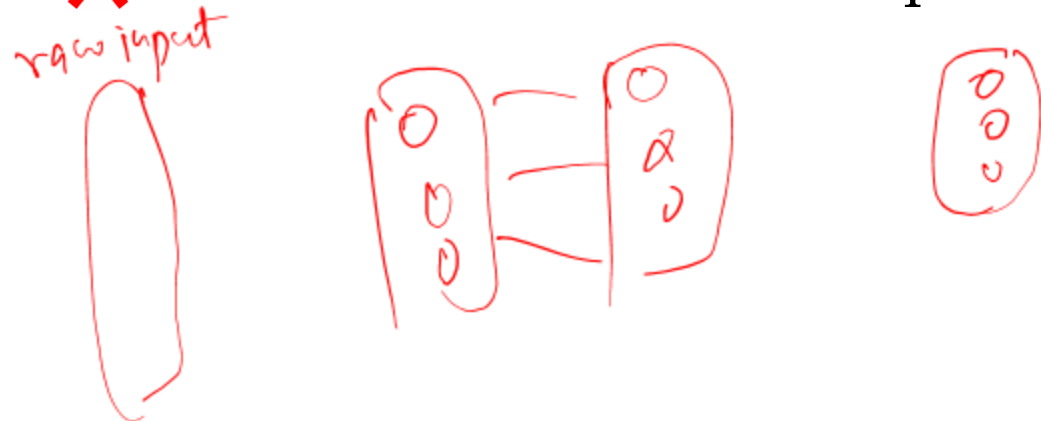
Image Classification

- **Neural networks for visual recognition**
 - ✗ 1. Maintain 2D structure logic
 - ✗ 2. Shift invariant (actually, equivariant)
 - ✗ 3. Consider local correlations first
 - ✗ 4. Hierarchically growing field of view
 - ✓ 5. Hierarchically progressing complexity
 - ✗ 6. Reasonable amount of params



Image Classification

- Neural networks for visual recognition
 - ✗ 1. Maintain 2D structure logic
 - ✗ 2. Shift invariant (actually, equivariant)
 - ✗ 3. Consider local correlations first
 - ✗ 4. Hierarchically growing field of view
 - ✓ 5. **Hierarchically progressing complexity (layers stacking)**
 - ✗ 6. Reasonable amount of params



input → out part.

Image Classification

- Neural networks for visual recognition
 - ✗ 1. Maintain 2D structure logic
 - ✗ 2. Shift invariant (actually, equivariant)
 - ✗ 3. Consider local correlations first
 - ✗ 4. Hierarchically growing field of view (local to global)
 - ✓ 5. **Hierarchically progressing complexity**
 - ✗ 6. Reasonable amount of params

Trainable feature hierarchy

- Hierarchy of representations with increasing levels of abstraction.
- Each stage is a kind of trainable feature transform.

Image recognition

- Pixel → edge → texton → motif → part → object

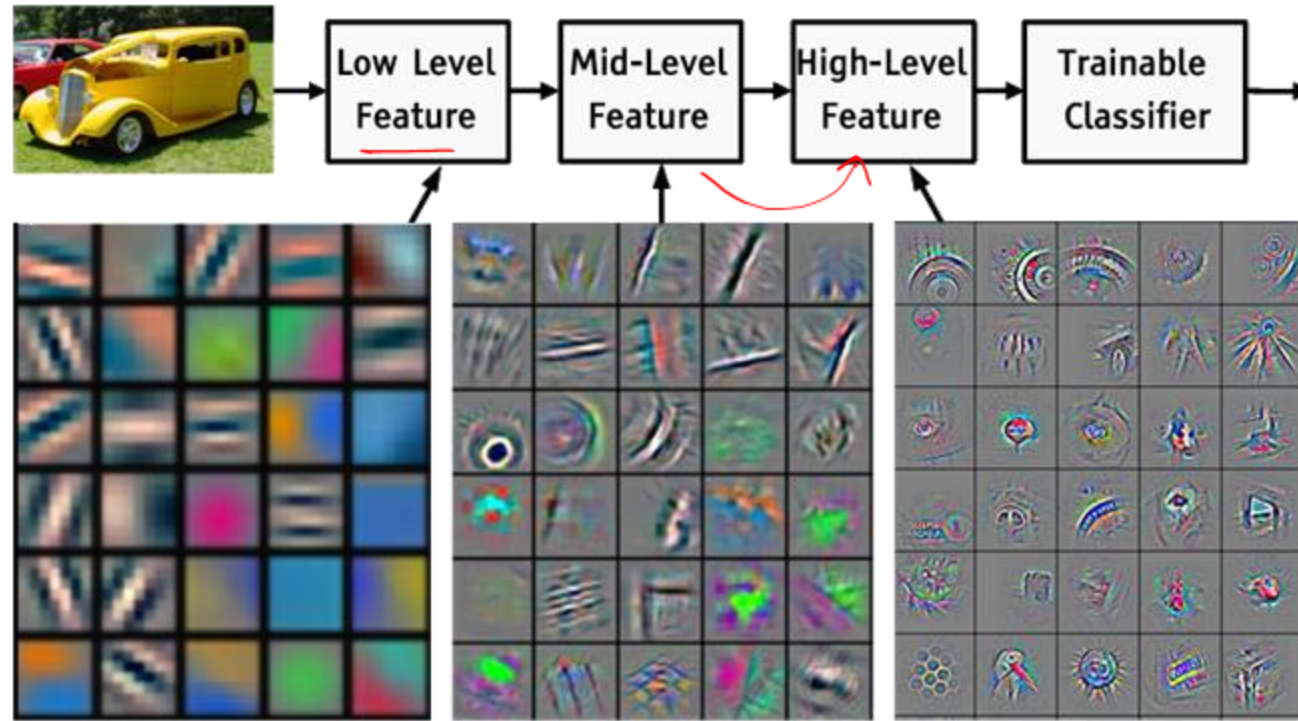
edges of
edges

Face

shapes

CNNs Motivation

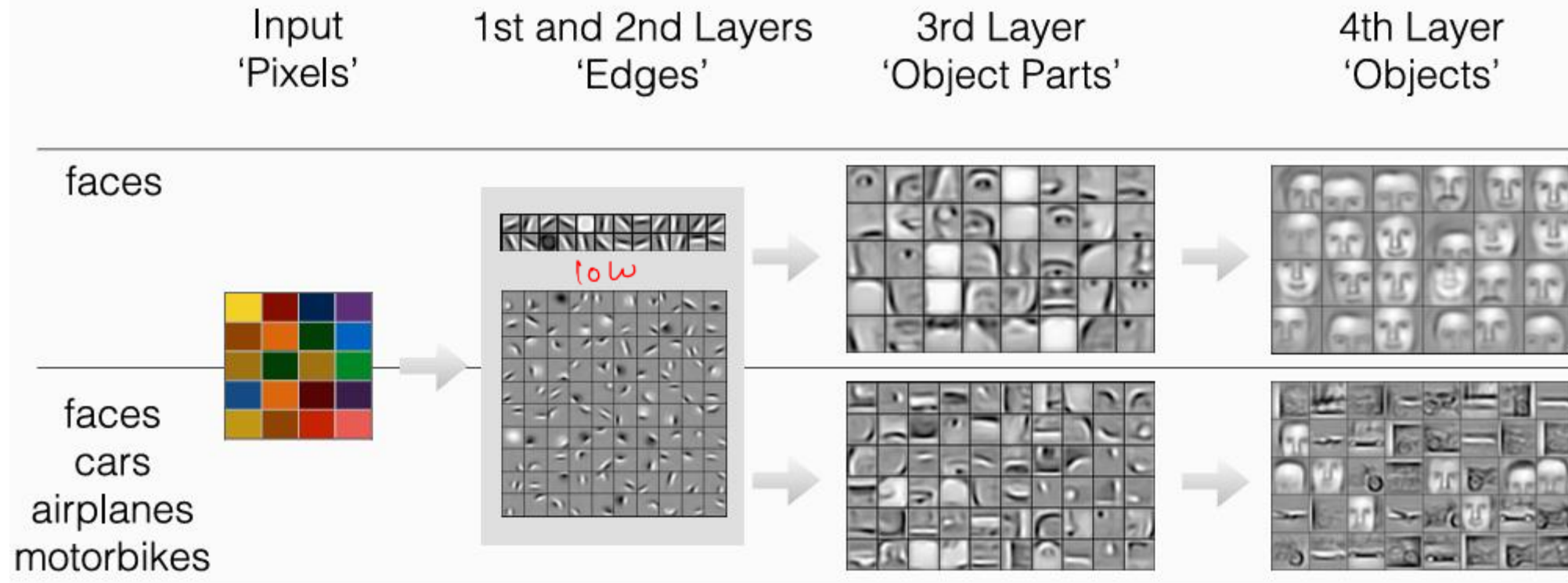
- Deep learning features extraction hierarchy:
 - It's **deep** if it has **more than one stage** of non-linear feature transformation



Feature visualization of convolutional net
trained on ImageNet from (Zeiler & Fergus, 2013)

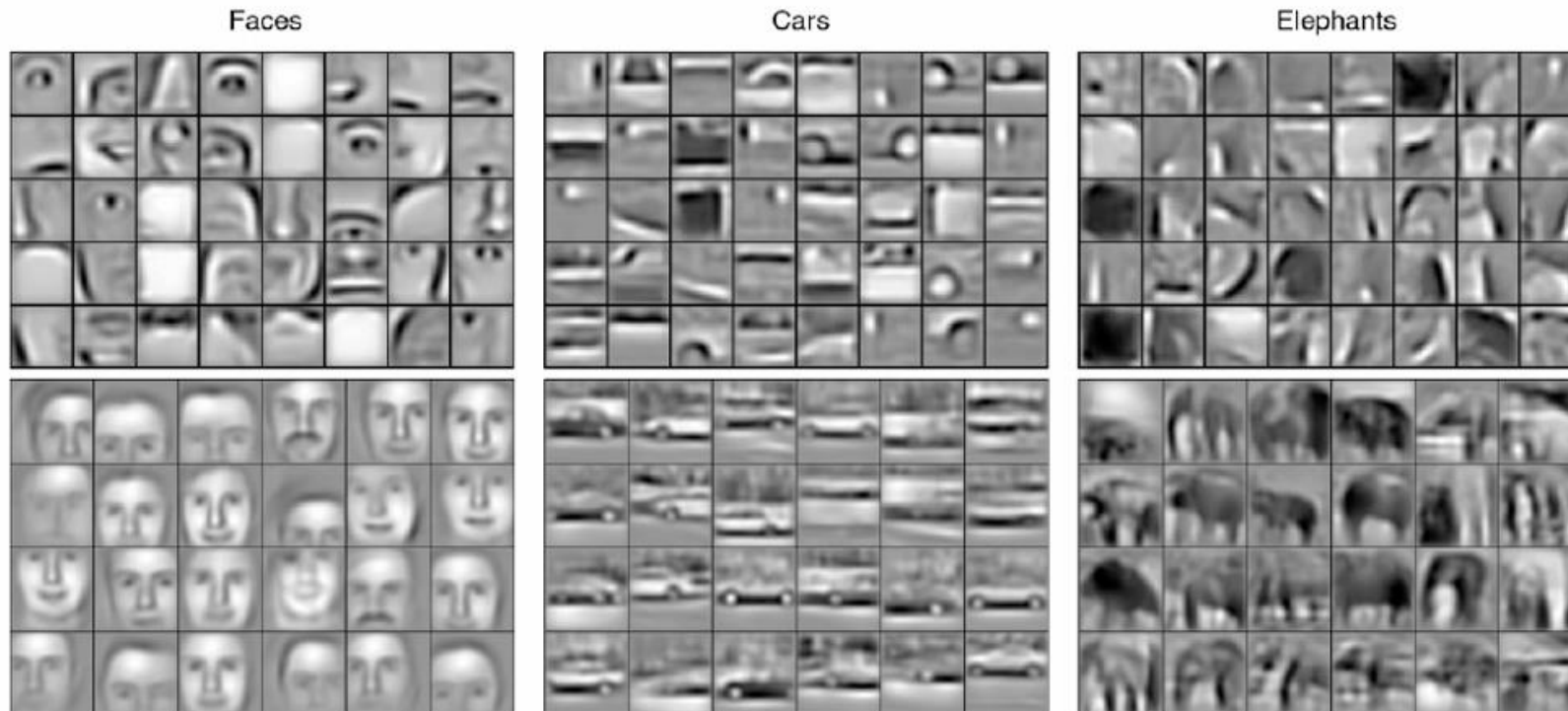
CNNs Motivation

- Each layer progressively extracts higher level features until the final layer. The more layers the network has, the higher-level features it will learn.



CNNs Motivation

- Mid- to high-level features are specific to the given task.
 - Low-level representations contain less specific features and can be shared across many different applications.
- more generic*



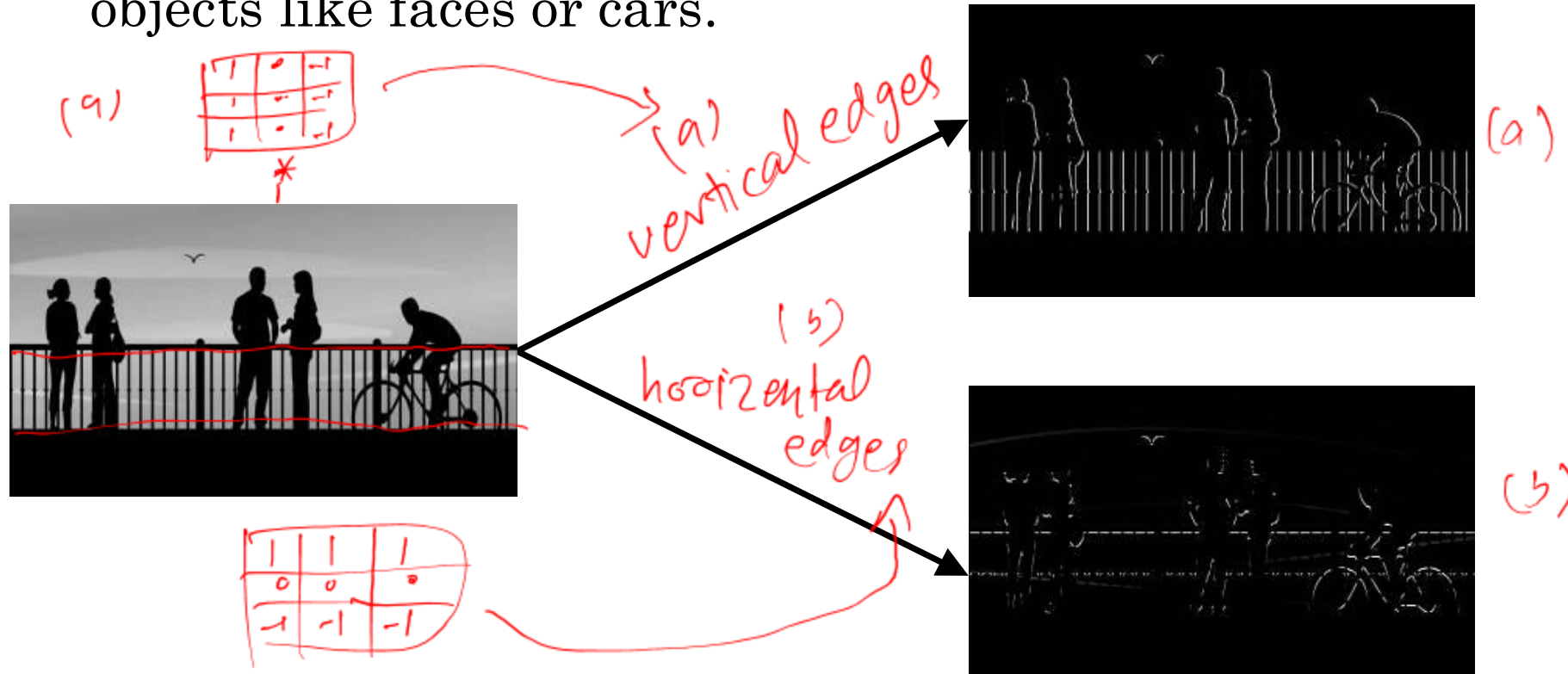
CNNs Motivation

- Humans recognise basic patterns first like edges, corners, and textures and then combine them into more complex features like eyes, and noses, or wheels, and finally into whole objects like faces or cars.



CNNs Motivation

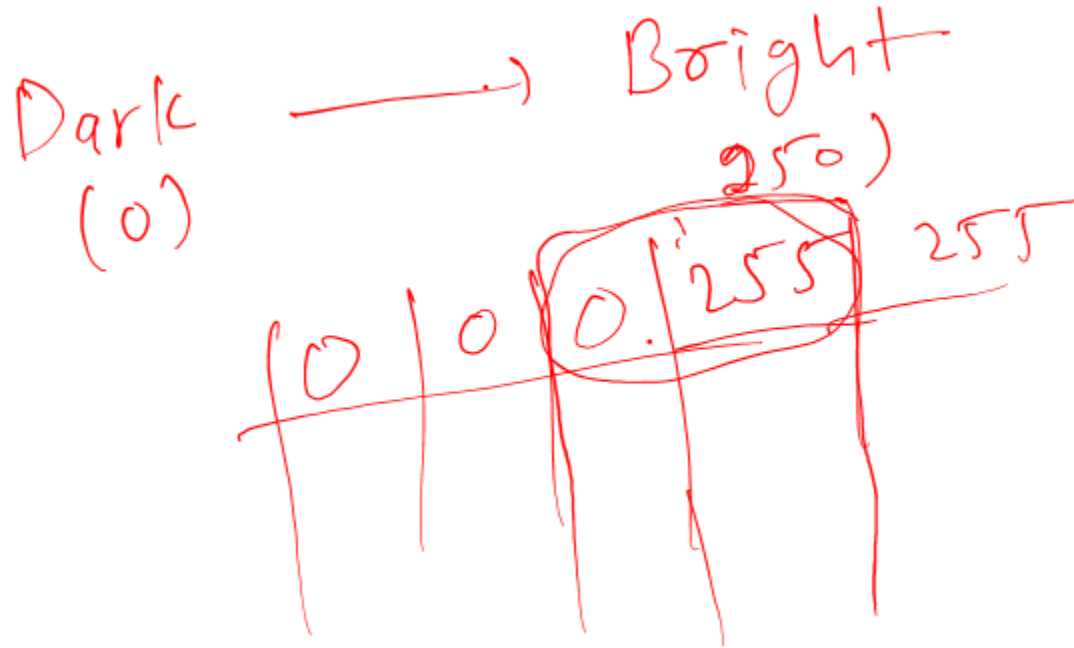
- Humans recognise basic patterns first like edges, corners, and textures and then combine them into more complex features like eyes, noses, or wheels, and finally into whole objects like faces or cars.





CNNs Motivation

- What is an Edge and how to detect edges?



CNNs Motivation

- What is an Edge and how to detect edges?

10 → brighter/white
0 → gray

I

a edge b

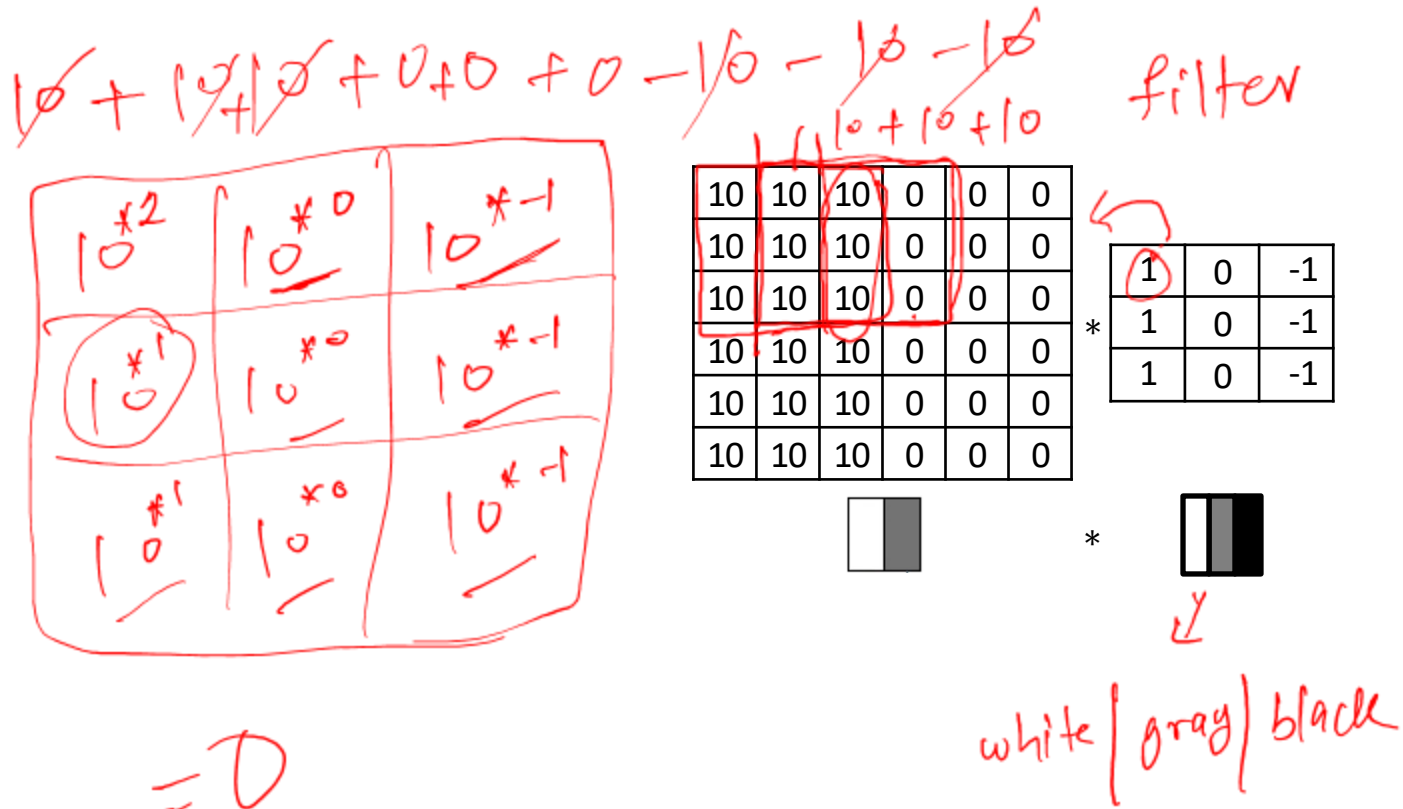
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0



Visual
representation →

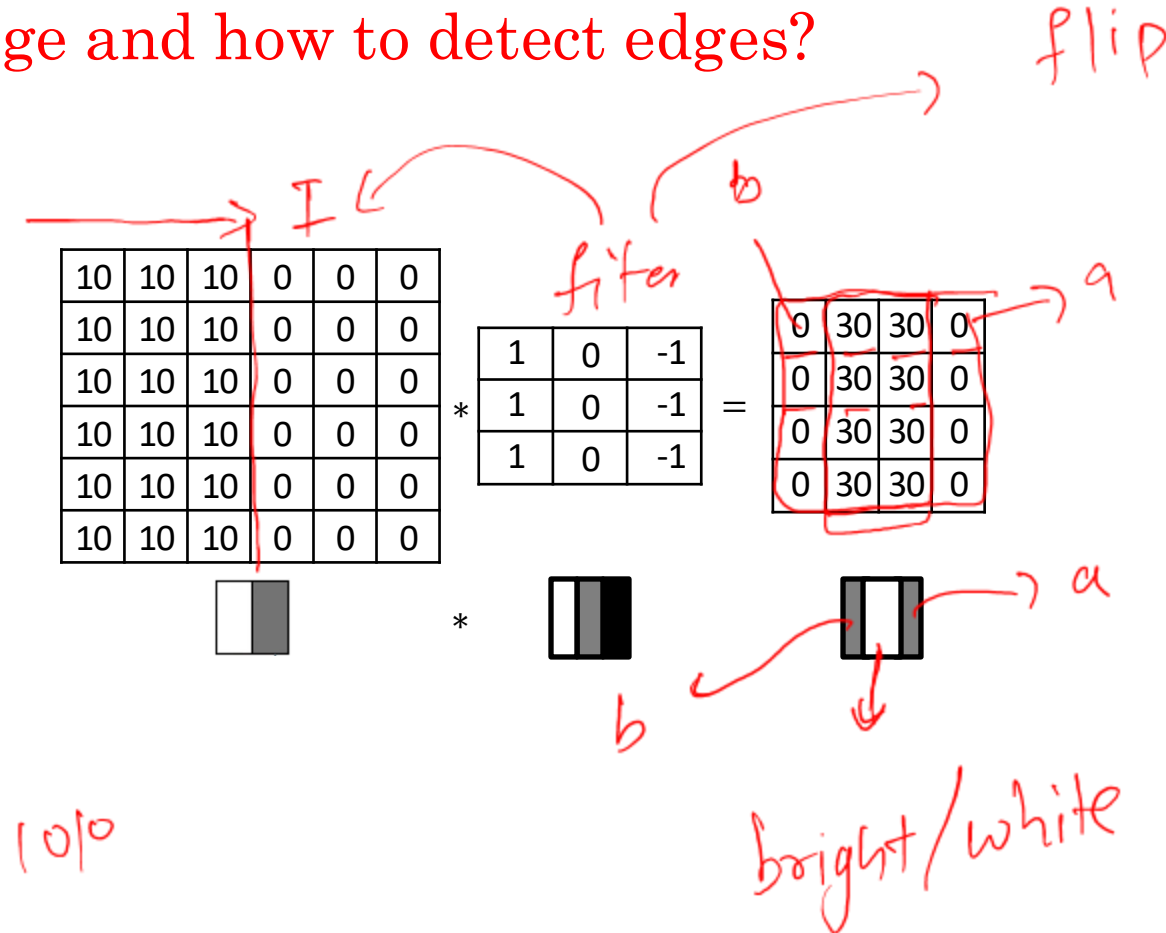
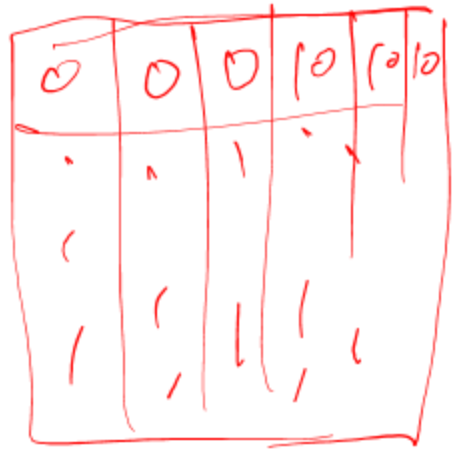
CNNs Motivation

- What is an Edge and how to detect edges?



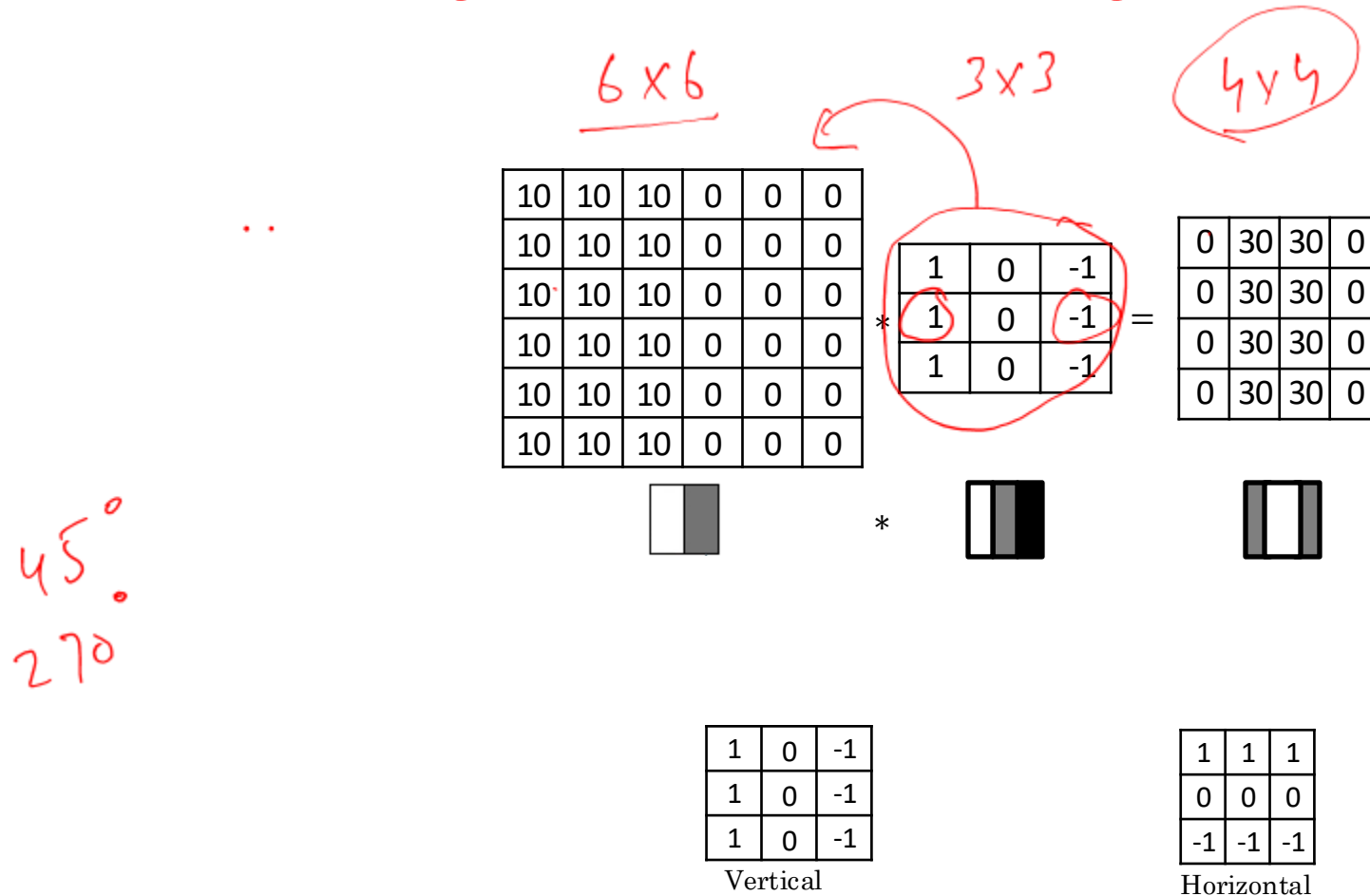
CNNs Motivation

- What is an Edge and how to detect edges?



CNNs Motivation

- What is an Edge and how to detect edges?



CNNs Motivation

- Edges detection filters

central

1	0	-1
2	0	-2
1	0	-1

Sobel filter

3	0	-3
10	0	-10
3	0	-3

Scharr filter



CNN Components: Convolution

- Learning to detect edges

3	0	1	2	7	4
1	5	8	9	3	1
2	7	2	5	1	3
0	1	3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

6x6

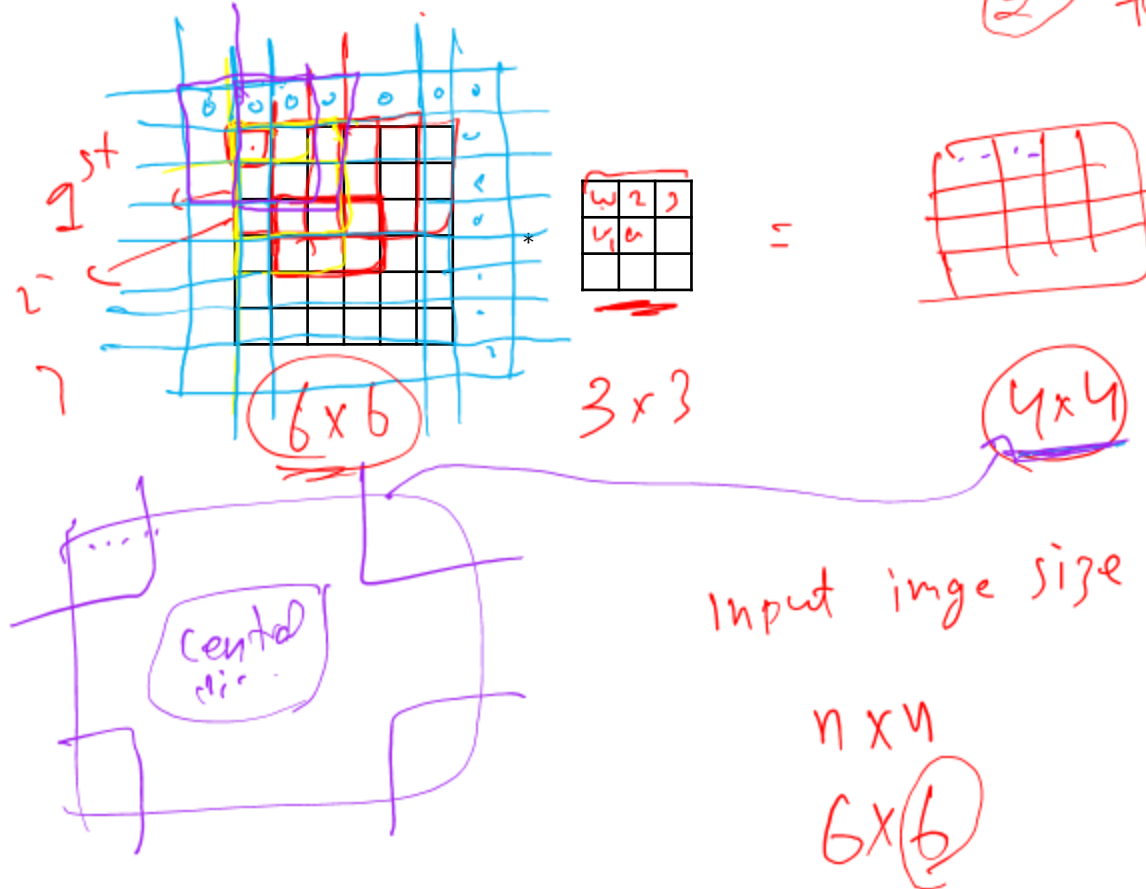
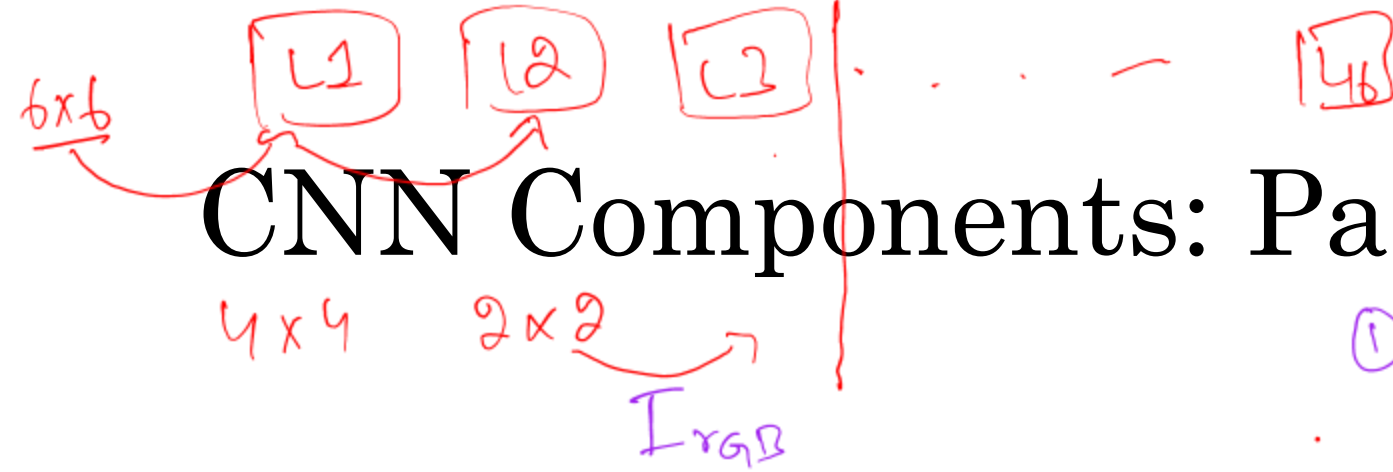
*

1	0	-1
1	0	-1
1	0	-1

3x3



CNN Components: Padding



- ① To not only consider the dominant side of conv. pixel.
 - ② throw away.
- padding.

$$n + 2p - f + 1$$

$$6 + 2(1) - 3 + 1$$

$$6 + 2 - 3 + 1$$

$$= 6 \times 6$$

size

output

$$n - f + 1 \times n - f + 1$$

$$6 - 3 + 1 \times 6 - 3 + 1$$

$$4 \times 4$$

Input image size

n x n

6x6

filter size

f x f

3x3





Handwritten notes illustrating a recursive formula for a sequence of numbers, likely related to a grid-based problem.

Top Section:

- A large grid (225x225) is shown with a 3x3 subgrid highlighted in the top-left corner. The 3x3 subgrid is labeled 3×3 .
- The operation is defined as: $225 \times 3 \times 3 = 222$.
- A smaller grid (222x222) is shown with a 3x3 subgrid highlighted in the top-left corner. The 3x3 subgrid is labeled 3×3 .
- The operation is defined as: $222 \times 3 \times 3 = 219$.

Bottom Section:

- A large grid (226x226) is shown with a 3x3 subgrid highlighted in the top-left corner. The 3x3 subgrid is labeled 3×3 .
- The operation is defined as: $226 \times 3 \times 3 = 224$.

Formulas:

$$2(n) + 2p - f + 1$$

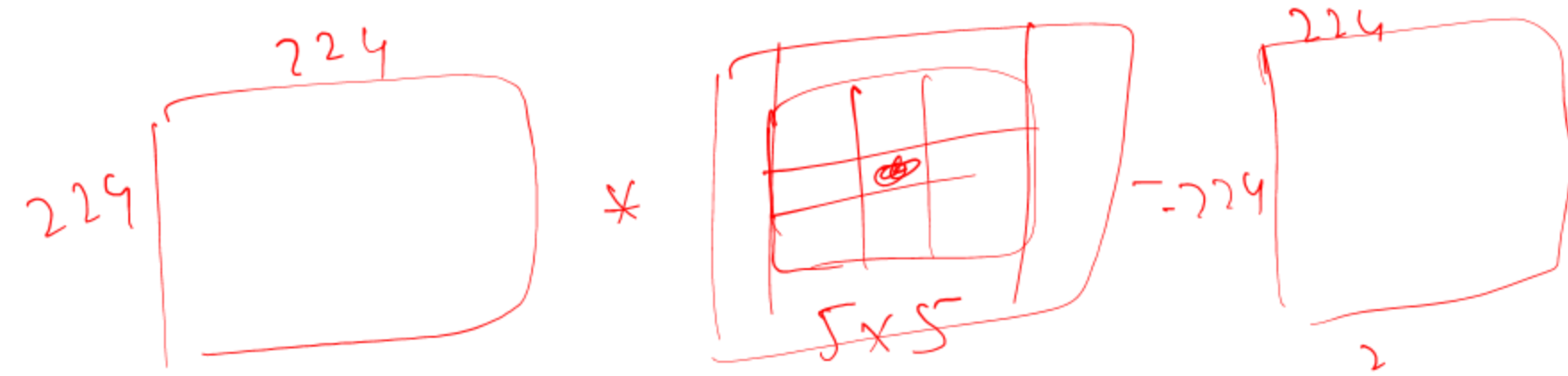
$$224 + 0 - 3 + 1 = 222$$

$$222 + 0 - 3 + 1 = 220$$

$$\vdots$$

$$224 + 2(1) - 3 + 1$$

$$226 - 3 + 1 = 224$$



① Valid $\rightarrow$ no padding. $\frac{3 \times 3}{.}$, $\underline{5 \times 5}$

② same $\rightarrow$ input size = output size

$$n + 2p - f + 1 = n$$

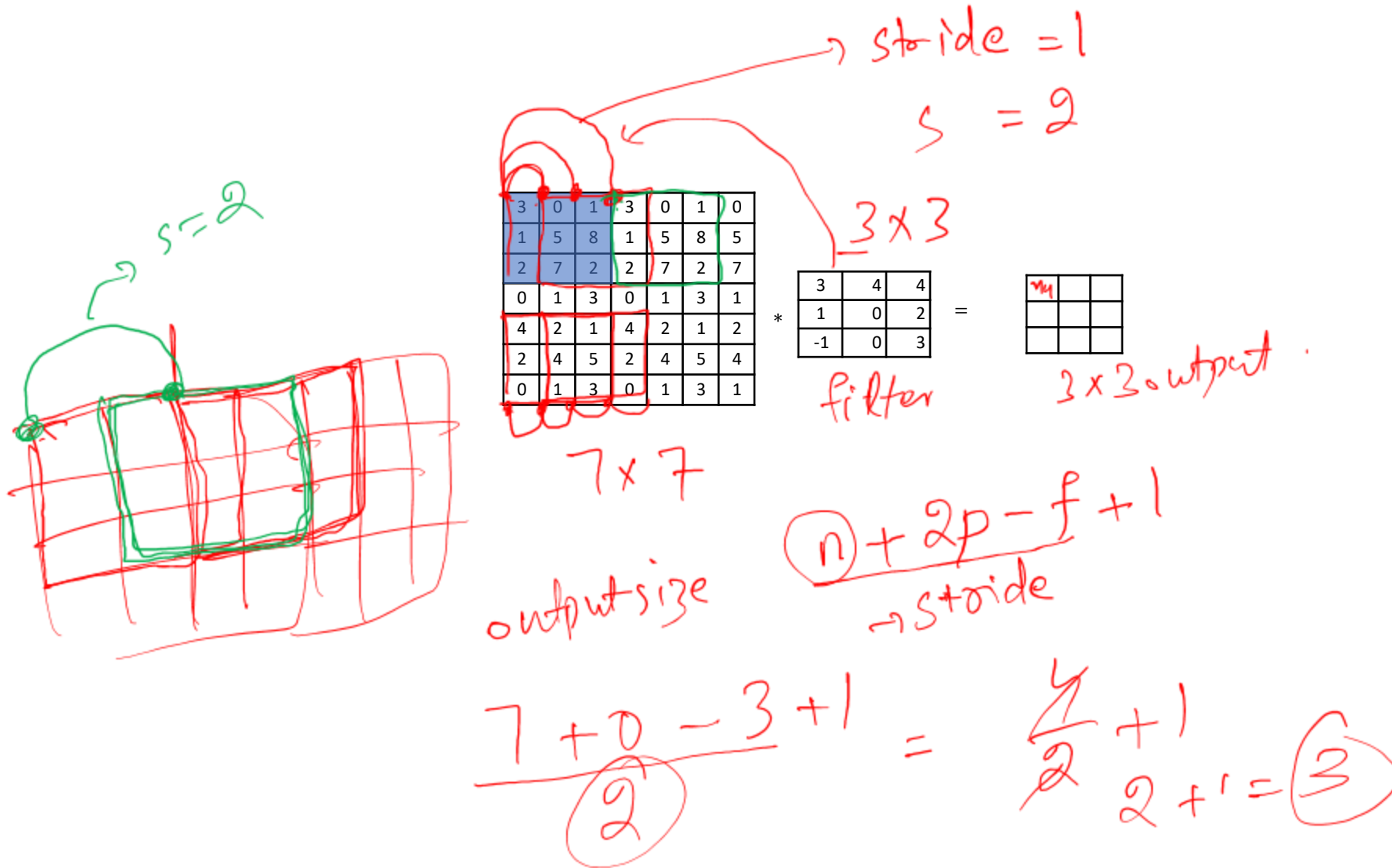
$$2p = f - 1$$

$$p = \frac{f - 1}{2}$$

$$p = \frac{5 - 1}{2} = \frac{4}{2} = 2$$



CNN Components: Stride



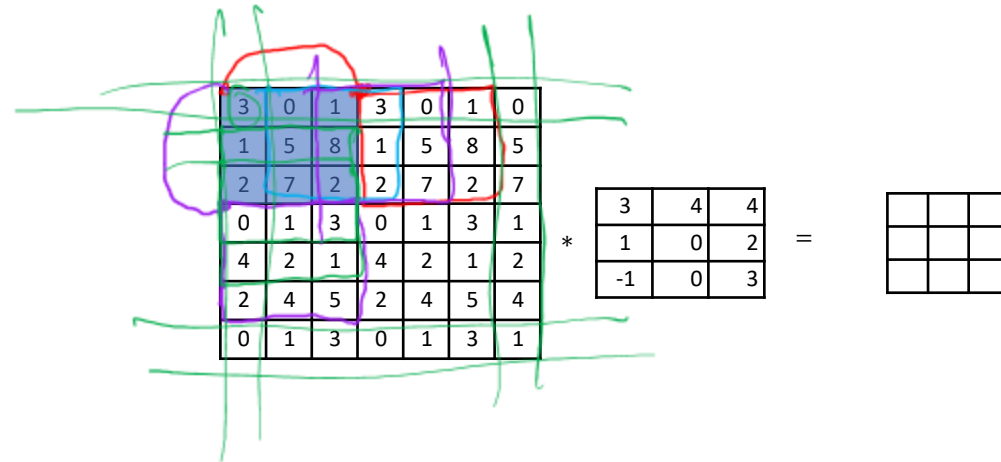
CNN Components: Stride

- Stride = 1?
- Stride > 1?

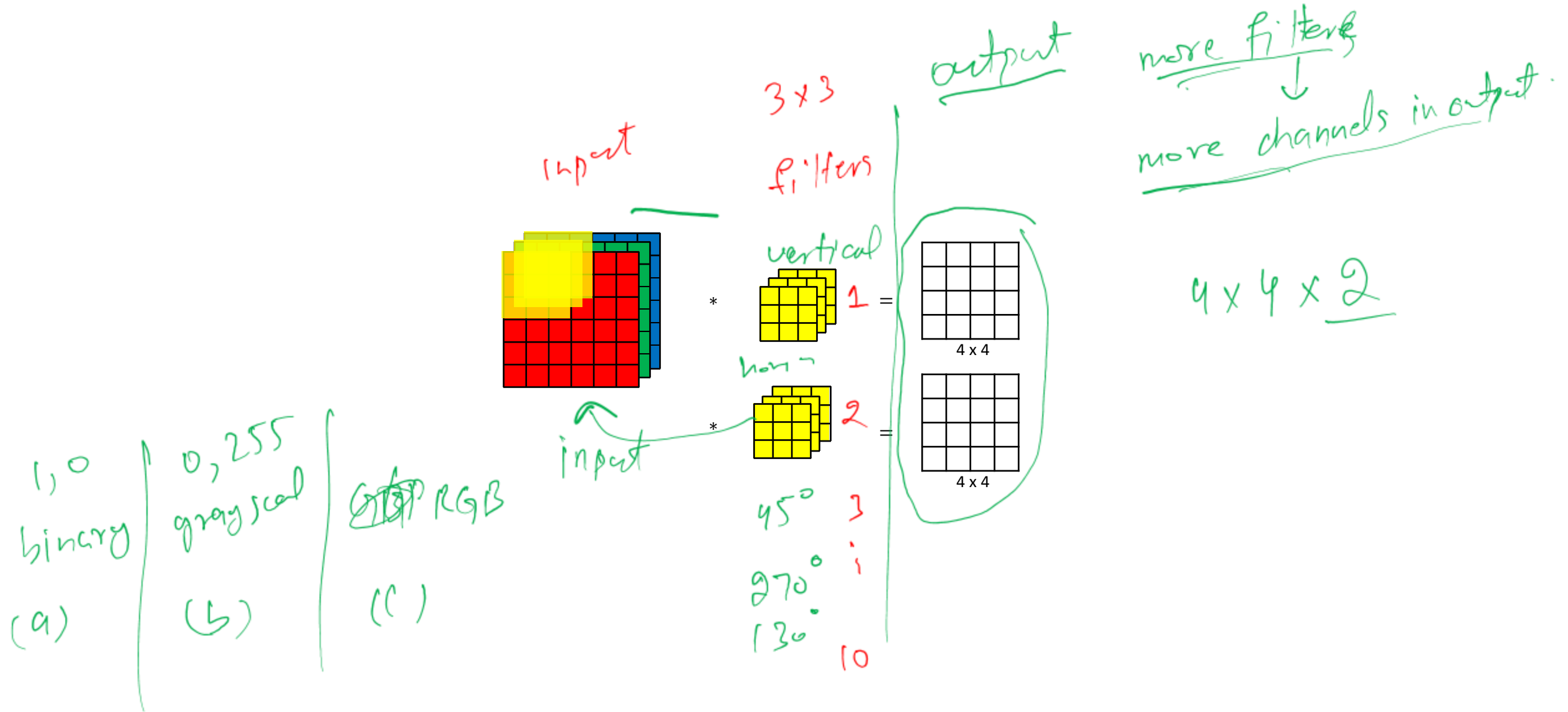
↙ flops

efficient.

$s = 2$



CNN Components: Convolution



CNN Components: Pooling

- **Pooling** reduces the spatial dimensions (width $\times$ height) of the feature map by summarising a small region of the feature map

0	1	3	2	1
0	2	9	1	1
1	3	2	3	
5	6	1	2	

4x4

max pooling.

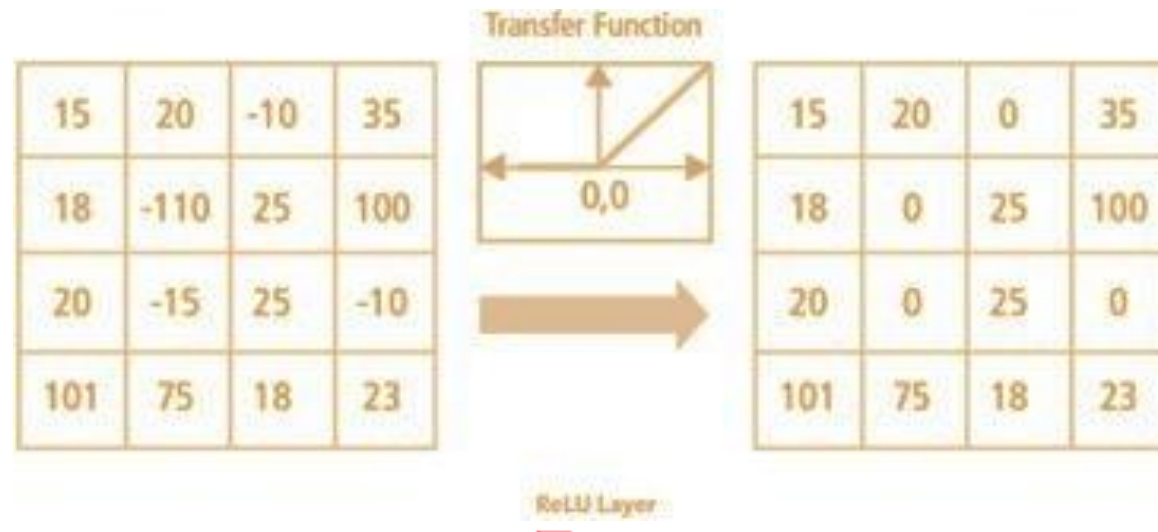
9	2
6	3

2x2

average pooling.

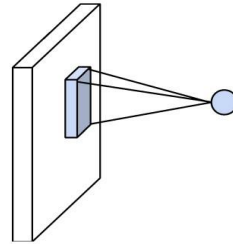
CNN Components: Activations

- As in fully-connected neural networks, we can apply an activation over convolutional layer outputs. It helps break linearity

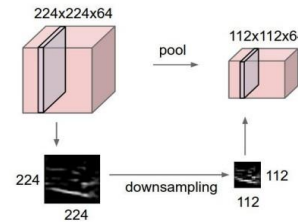


Components of a CNN Model

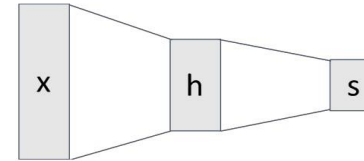
Convolution Layers



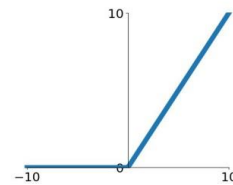
Pooling Layers



Fully-Connected Layers



Activation Function



Normalization

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \epsilon}}$$

CNN Models

- How is the problem of huge parameters solved?



CNN Models

- How is the problem of huge parameters solved?
- Let's compute the number of parameters for 10 filters of size 3 by 3 by 3.

$3 \times 3 \times 3 = 27 \text{ parameter}$
 $+ 1 = 28 \text{ param}$ ← 
 28
 ⋮

 ⋮

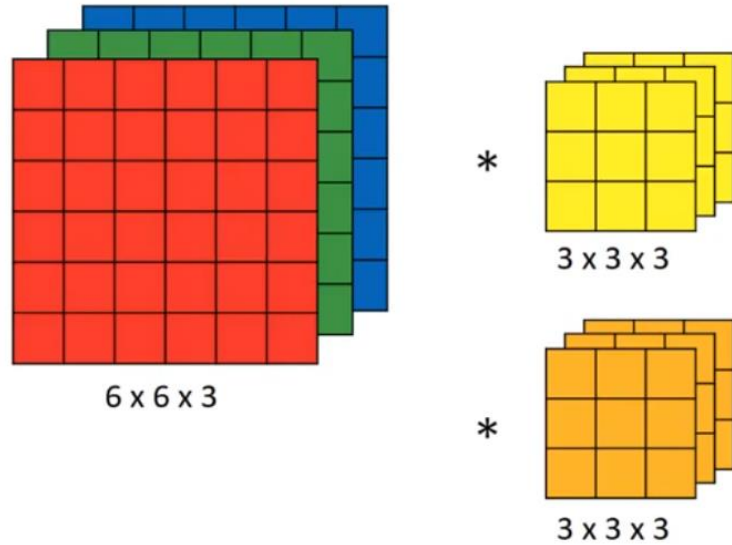

1st filter

2

10th filter

28×10
 $= 280$
 224×224
 $128, 128$

Designing a CNN Layer



References

- https://web.eecs.umich.edu/~justincj/slides/eecs498/WI2022/598_WI2022_lecture02.pdf
- https://web.eecs.umich.edu/~justincj/slides/eecs498/WI2022/598_WI2022_lecture03.pdf
- <https://towardsdatascience.com/training-deep-neural-networks-9fdb1964b964>
- <https://www.charles-deledalle.fr/pages/teaching.php>
- https://www.charles-deledalle.fr/pages/files/ucsd_ece285_mlip/3_deep.pdf